Donkey Kong Repair Lab

Complete chat history with Claude (Anthropic), October 22, 2026. Covers the Barrel ladder-descent fix and the theme_color, on_barrel_jumped and score_multiplier features.

YOU

I'm doing a lab where I fix and extend a Python Pygame Donkey Kong game. Below are the README and the original game.py. Don't change anything yet. Briefly explain how the Barrel class decides to take a ladder, and what the three empty functions (theme_color, on_barrel_jumped, score_multiplier) are supposed to do.

Attached files: game.py, README (2).md

CLAUDE

How a Barrel decides to take a ladder

Each frame, while a barrel is rolling along a platform, it moves horizontally, snaps its y to the slope, and then loops over every ladder. For each one it checks three things: the ladder's *upper* platform is the one the barrel is on, the barrel's `x` is within 3 pixels of the ladder's `x`, and that ladder isn't already in the barrel's `skip` set.

If all three hold, the barrel adds that ladder to `skip` (so it only ever gets one chance per ladder, even if it lingers within 3 px for a few frames) and then rolls a dice: `random.random() < 0.7`. If that passes, the barrel locks onto the ladder, snaps its `x` to the ladder, and on later frames drops straight down at 90 px/s until it reaches the lower platform, where it resumes rolling. If the roll fails, the barrel just keeps rolling past the ladder.

That `0.7` is the bug from Task 1: it makes barrels descend about 70% of the time, whereas the README wants roughly 30%. Nothing else in the logic needs to change for that task.

The three empty functions

All three currently contain `pass`, so they return `None`, which the calling code treats as "use the default."

- `theme_color(score)` is called every frame as `screen.fill(theme_color(score) or BG)`. It takes the current integer score and should return an `(r, g, b)` tuple for the background, or `None` to keep the default navy `(15, 15, 25)`. The README suggests shifting toward a warmer colour as score climbs, or flashing a colour after a big gain.
- `on_barrel_jumped(player, barrel)` is called exactly once per barrel, on the first frame the player is airborne directly above it, right after the 100 points are added. It gets the `Player` and the `Barrel`, and its return value is ignored, so it exists only for side effects, such as spawning a floating "+100" label at the barrel's position or playing a sound. Since it's called once and the game has no existing effects list, a floating label would need some state stored outside the function (a module-level list) plus drawing code, which means touching `draw_scene` or the main loop.
- `score_multiplier(score)` is called as `score += int(100 * (score_multiplier(score) or 1))`. It receives the score *before* the bonus and should return a number, or `None` for 1x. For example, return `2` once the score passes a threshold. One small gotcha: returning `0` would also fall through to the default 1x because of the `or 1`.

I haven't changed anything. When you're ready, tell me which task to start with.

YOU

Task 1: barrels take the first ladder they reach about 7 times out of 10, but they should only descend about 3 times out of 10. Find the probability check in the Barrel class and fix it. Show only the changed lines and explain why.

CLAUDE

Changed line (in `Barrel.update`, the ladder loop at the end):

```
# before
if random.random() < 0.7:

# after
if random.random() < 0.3:
```

Why: `random.random()` returns a float uniformly distributed in [0, 1). The condition `< p` is true with probability `p`, so `< 0.7` sends a barrel down the ladder about 70% of the time. Changing the threshold to `0.3` makes it true about 30% of the time, which matches the README's "roughly 3 out of 10."

The `skip` set already guarantees each barrel rolls this dice only once per ladder, so no other change is needed. Each barrel gets a single 30% chance at each ladder it passes, and the rest keep rolling along the platform.

YOU

Task 2: implement theme_color(score). It's called as screen.fill(theme_color(score) or BG) with BG = (15, 15, 25). Return None at score 0, then shift the background toward a warmer colour as the score climbs, capped at a maximum. Give me the complete function and any constants it needs.

CLAUDE

Add the constants right below `BG`:

```
WARM_BG = (95, 35, 20)
WARM_SCORE_CAP = 2000
```

And replace the empty function:

```
def theme_color(score):
    """Return an (r, g, b) background colour for the current score, or None for the default."""
    if score <= 0:
        return None
    t = min(score / WARM_SCORE_CAP, 1.0)
    return tuple(int(base + (warm - base) * t) for base, warm in zip(BG, WARM_BG))
```

How it works: `t` is the fraction of the way from the default background to the warm one. At score 0 the function returns `None`, so `or BG` keeps the navy. At score 1000 `t` is 0.5, so each colour channel is halfway between navy and the warm brown-red. From 2000 up, `min(..., 1.0)` holds `t` at 1.0, so the background stops changing at `WARM_BG`.

Each channel is blended separately: `base + (warm - base) * t`. At `t = 0` that gives the original value, and at `t = 1` it gives the warm value. `int()` is needed because `screen.fill` wants whole-number colour values.

To tune it, make `WARM_BG` brighter for a stronger effect, or lower `WARM_SCORE_CAP` so the colour maxes out sooner. I kept `WARM_BG` fairly dark so the red platforms, yellow ladders and barrels stay readable against it.

YOU

Task 3: implement on_barrel_jumped(player, barrel). It's called once when the player clears a barrel. Spawn a short-lived floating "+100" label at the barrel's position that rises and fades out. Show me every change needed (the function, where the labels are stored, where they're updated and drawn in the main loop) and keep the changes small.

CLAUDE

Four small changes, all in `game.py`.

1. Storage and constants (below `BG` and the other constants)

```
FLOATERS = []
FLOAT_LIFE, FLOAT_RISE = 0.8, 50
```

2. The function

```
def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    FLOATERS.append([barrel.pos.x, barrel.pos.y, 0.0])
```

3. Update, in `main()`, right after the `if state == "play":` block and before `draw_scene(...)`:

```
        for f in FLOATERS:
            f[2] += dt
        FLOATERS[:] = [f for f in FLOATERS if f[2] < FLOAT_LIFE]
```

4. Draw, in `draw_scene`, just before the `hud = ...` line:

```
    for x, y, age in FLOATERS:
        label = font.render("+100", True, (255, 255, 120))
        label.set_alpha(int(255 * (1 - age / FLOAT_LIFE)))
        screen.blit(label, label.get_rect(center=(x, y - age * FLOAT_RISE)))
```

How it works: Each label is a small list `[x, y, age]` holding where the barrel was when you cleared it and how long the label has existed. The update step adds the frame time to every label's age and drops any older than 0.8 seconds. The draw step uses the age two ways. The label sits `age * 50` pixels above its starting point, so it floats upward, and its opacity is `255 * (1 - age / 0.8)`, so it fades from solid to invisible as it reaches the end of its life.

The update step sits outside the `if state == "play":` block on purpose, so labels still expire if you win or lose while one is on screen.

Fading relies on `set_alpha` working on text surfaces, which it does in pygame 2. If you're on pygame 1.9, the label won't fade.

The label always reads "+100". Once you add the multiplier in Task 4, the real bonus can be 200, so you may want to store the actual points in the label instead.

YOU

Task 4: implement score_multiplier(score). It's called as score += int(100 * (score_multiplier(score) or 1)). Return 2 once the score reaches 500, otherwise None. The floating label from Task 3 should show the real points awarded (+100 or +200). Show the changes needed.

CLAUDE

Four changes. The function signatures stay as the README defines them.

1. `score_multiplier`

```
def score_multiplier(score):
    """Return a multiplier applied to points earned from clearing a barrel, or None for the default 1x."""
    if score >= 500:
        return 2
    return None
```

2. `Barrel.__init__`: add one line next to `self.scored = False`:

```
        self.points = 0
```

3. Main loop: replace the scoring line:

```
# before
                    score += int(100 * (score_multiplier(score) or 1))
                    on_barrel_jumped(player, barrel)

# after
                    barrel.points = int(100 * (score_multiplier(score) or 1))
                    score += barrel.points
                    on_barrel_jumped(player, barrel)
```

4. Task 3 code: the label now stores the points.

```
def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    FLOATERS.append([barrel.pos.x, barrel.pos.y, 0.0, barrel.points])
```

In `draw_scene`, replace the floater loop:

```
    for x, y, age, points in FLOATERS:
        label = font.render(f"+{points}", True, (255, 255, 120))
        label.set_alpha(int(255 * (1 - age / FLOAT_LIFE)))
        screen.blit(label, label.get_rect(center=(x, y - age * FLOAT_RISE)))
```

The update loop in `main()` needs no change, since it only uses `f[2]`.

How it works: `on_barrel_jumped` doesn't receive the awarded points, so the main loop computes them once, stores them on the barrel as `barrel.points`, and adds that value to the score. The function then copies `barrel.points` into the label, so the label always matches what was actually added.

The multiplier is checked against the score *before* the bonus. The jump that takes you from 400 to 500 still pays 100 and shows "+100", and every jump after that pays 200 and shows "+200". The win bonus of 1000 isn't affected, since it doesn't go through the multiplier.

YOU

Give me the complete final game.py as one file, and tell me how to run it and what to look for to confirm all four tasks work.

CLAUDE

The complete `game.py` is attached. It compiles, and a diff against your original shows only the changes from the four tasks, with nothing else touched. The full file is reproduced in the appendix at the end of this document.

How to run it

Put `game.py` in your `donkey_kong/` folder and run:

```
pip install pygame
python game.py
```

Controls: Left/Right to move, Space to jump, Up/Down to grab a ladder, R to reset.

What to look for

1. **Task 1, ladder descent.** Watch the barrels from the top platform for a minute or two. Most should roll straight past each ladder, and only about 3 in 10 should drop down it. Before the fix, nearly every barrel took the first ladder it reached. For your "after" video, the top platform should visibly stay populated.

2. **Task 2, background colour.** At score 0 the background is the original navy. After your first barrel jump (score 100), it shifts very slightly warmer. Around 1000 points it's a noticeable dark brown-red, and it stops changing at 2000. Pressing R resets the score and the background goes back to navy.

3. **Task 3, floating label.** Jump over a barrel while it passes under you. A yellow label should appear at the barrel's position, rise about 40 pixels, and fade out in under a second. If it never appears, you probably jumped too early or too late. The player has to be in the air with the barrel directly underneath.

4. **Task 4, multiplier.** The first labels read "+100". Once the score reaches 500, labels read "+200" and the score climbs by 200 per jump. The jump that takes you from 400 to 500 still shows "+100", because the multiplier checks the score before the bonus.

For the 10-second "after" video, a jump gets you from 0 to 500 slowly, so it helps to clear several barrels in a row and show the label change from +100 to +200. Reaching the princess adds 1000 points to the score, but that bonus doesn't use the multiplier.

Claude also produced the file game.py (see appendix).

YOU

give this entire chat history as a pdf document

Appendix: final game.py

```
import random
import pygame

WIDTH, HEIGHT = 800, 600
PLAYER_W, PLAYER_H = 20, 28
BARREL_R, BARREL_SPEED = 10, 140
WALK_SPEED, CLIMB_SPEED, JUMP_SPEED, GRAVITY = 170, 110, 380, 900
BG = (15, 15, 25)
WARM_BG = (95, 35, 20)
WARM_SCORE_CAP = 2000
FLOATERS = []
FLOAT_LIFE, FLOAT_RISE = 0.8, 50

# (x_left, x_right, y_at_left, y_at_right)
PLATFORMS = [
    (0, 800, 570, 570),
    (0, 740, 480, 505),
    (60, 800, 410, 385),
    (0, 740, 310, 330),
    (0, 600, 200, 200),
]
# (x, lower platform index, upper platform index)
LADDERS = [(650, 0, 1), (120, 1, 2), (640, 2, 3), (140, 3, 4)]
KONG_POS = (60, 200)
PRINCESS_POS = (540, 200)


def platform_y(platform, x):
    x1, x2, y1, y2 = platform
    return y1 + (y2 - y1) * (x - x1) / (x2 - x1)


def theme_color(score):
    """Return an (r, g, b) background colour for the current score, or None for the default."""
    if score <= 0:
        return None
    t = min(score / WARM_SCORE_CAP, 1.0)
    return tuple(int(base + (warm - base) * t) for base, warm in zip(BG, WARM_BG))


def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    FLOATERS.append([barrel.pos.x, barrel.pos.y, 0.0, barrel.points])


def score_multiplier(score):
    """Return a multiplier applied to points earned from clearing a barrel, or None for the default 1x."""
    if score >= 500:
        return 2
    return None


class Player:
    def __init__(self):
        self.reset()

    def reset(self):
        self.pos = pygame.Vector2(40, PLATFORMS[0][2])
        self.vel = pygame.Vector2()
        self.on_ground = True
        self.ladder = None

    def center(self):
        return pygame.Vector2(self.pos.x, self.pos.y - PLAYER_H / 2)

    def find_ladder(self, going_up):
        for index, (lx, lower, upper) in enumerate(LADDERS):
            top = platform_y(PLATFORMS[upper], lx)
            bottom = platform_y(PLATFORMS[lower], lx)
            if abs(self.pos.x - lx) > 10:
                continue
            if going_up and top + 2 < self.pos.y <= bottom + 3:
                return index
            if not going_up and top - 3 <= self.pos.y < bottom - 2:
                return index
```

```
        return None

    def jump(self):
        if self.on_ground and self.ladder is None:
            self.vel.y = -JUMP_SPEED
            self.on_ground = False

    def update(self, dt, keys):
        move = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        vertical = keys[pygame.K_DOWN] - keys[pygame.K_UP]
        if self.ladder is None and vertical:
            found = self.find_ladder(going_up=vertical < 0)
            if found is not None:
                self.ladder = found
                self.pos.x = LADDERS[found][0]
                self.vel.update(0, 0)
                self.on_ground = False
        if self.ladder is not None:
            self.climb(dt, vertical)
            return
        self.pos.x = max(10, min(WIDTH - 10, self.pos.x + move * WALK_SPEED * dt))
        if self.on_ground and self.vel.y >= 0:
            for plat in PLATFORMS:
                if plat[0] <= self.pos.x <= plat[1]:
                    py = platform_y(plat, self.pos.x)
                    if abs(py - self.pos.y) <= 8:
                        self.pos.y = py
                        return
            self.on_ground = False
        previous = self.pos.y
        self.vel.y = min(self.vel.y + GRAVITY * dt, 700)
        self.pos.y += self.vel.y * dt
        if self.vel.y >= 0:
            for plat in PLATFORMS:
                if plat[0] <= self.pos.x <= plat[1]:
                    py = platform_y(plat, self.pos.x)
                    if previous <= py + 3 and self.pos.y >= py:
                        self.pos.y = py
                        self.vel.y = 0
                        self.on_ground = True
                        break

    def climb(self, dt, vertical):
        lx, lower, upper = LADDERS[self.ladder]
        self.pos.y += vertical * CLIMB_SPEED * dt
        top = platform_y(PLATFORMS[upper], lx)
        bottom = platform_y(PLATFORMS[lower], lx)
        if self.pos.y <= top:
            self.pos.y = top
            self.ladder = None
            self.on_ground = True
        elif self.pos.y >= bottom:
            self.pos.y = bottom
            self.ladder = None
            self.on_ground = True


class Barrel:
    def __init__(self):
        self.plat = 4
        self.direction = 1
        self.ladder = None
        self.vy = 0
        self.scored = False
        self.points = 0
        self.skip = set()
        self.pos = pygame.Vector2(KONG_POS[0] + 40, PLATFORMS[4][2] - BARREL_R)

    def update(self, dt):
        if self.ladder is not None:
            lx, lower, _ = LADDERS[self.ladder]
            self.pos.y += 90 * dt
            floor = platform_y(PLATFORMS[lower], lx)
            if self.pos.y + BARREL_R >= floor:
                self.plat = lower
                self.ladder = None
            return
        if self.plat is None:
```

```
            previous = self.pos.y + BARREL_R
            self.vy += GRAVITY * dt
            self.pos.y += self.vy * dt
            feet = self.pos.y + BARREL_R
            for index, plat in enumerate(PLATFORMS):
                if plat[0] <= self.pos.x <= plat[1]:
                    py = platform_y(plat, self.pos.x)
                    if previous <= py + 2 and feet >= py:
                        self.plat, self.vy = index, 0
                        break
            return
        plat = PLATFORMS[self.plat]
        slope = plat[3] - plat[2]
        if slope:
            self.direction = 1 if slope > 0 else -1
        self.pos.x += self.direction * BARREL_SPEED * dt
        if not plat[0] <= self.pos.x <= plat[1]:
            self.plat, self.vy = None, 0
            return
        self.pos.y = platform_y(plat, self.pos.x) - BARREL_R
        for index, (lx, _, upper) in enumerate(LADDERS):
            if upper == self.plat and abs(self.pos.x - lx) < 3 and index not in self.skip:
                self.skip.add(index)
                if random.random() < 0.3:
                    self.ladder = index
                    self.pos.x = lx


def draw_scene(screen, font, player, barrels, score, lives, state):
    screen.fill(theme_color(score) or BG)
    for x1, x2, y1, y2 in PLATFORMS:
        pygame.draw.line(screen, (210, 55, 55), (x1, y1), (x2, y2), 8)
    for lx, lower, upper in LADDERS:
        top = platform_y(PLATFORMS[upper], lx)
        bottom = platform_y(PLATFORMS[lower], lx)
        pygame.draw.line(screen, (220, 190, 80), (lx - 8, top), (lx - 8, bottom), 3)
        pygame.draw.line(screen, (220, 190, 80), (lx + 8, top), (lx + 8, bottom), 3)
        for y in range(int(top) + 8, int(bottom), 12):
            pygame.draw.line(screen, (220, 190, 80), (lx - 8, y), (lx + 8, y), 2)
    pygame.draw.rect(screen, (150, 90, 40), (KONG_POS[0] - 25, KONG_POS[1] - 50, 50, 50))
    pygame.draw.circle(screen, (255, 150, 200), (PRINCESS_POS[0], PRINCESS_POS[1] - 14), 10)
    for barrel in barrels:
        pygame.draw.circle(screen, (170, 100, 40), barrel.pos, BARREL_R)
    body = pygame.Rect(0, 0, PLAYER_W, PLAYER_H)
    body.midbottom = (player.pos.x, player.pos.y)
    pygame.draw.rect(screen, (50, 180, 240), body)
    for x, y, age, points in FLOATERS:
        label = font.render(f"+{points}", True, (255, 255, 120))
        label.set_alpha(int(255 * (1 - age / FLOAT_LIFE)))
        screen.blit(label, label.get_rect(center=(x, y - age * FLOAT_RISE)))
    hud = font.render(f"Score {score}   Lives {lives}   R = reset", True, (240, 240, 240))
    screen.blit(hud, (10, 8))
    if state != "play":
        text = "YOU WIN! Press R" if state == "win" else "GAME OVER - Press R"
        label = font.render(text, True, (255, 255, 120))
        screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Donkey Kong")
    clock = pygame.time.Clock()
    font = pygame.font.Font(None, 28)
    player, barrels = Player(), []
    score, lives, state, spawn_timer = 0, 3, "play", 1.0
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
                player.jump()
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                player.reset()
                barrels.clear()
                score, lives, state = 0, 3, "play"
```

```
        if state == "play":
            player.update(dt, pygame.key.get_pressed())
            spawn_timer -= dt
            if spawn_timer <= 0:
                barrels.append(Barrel())
                spawn_timer = random.uniform(1.8, 3.2)
            for barrel in barrels:
                barrel.update(dt)
                hit_range = BARREL_R + PLAYER_W / 2
                if player.center().distance_squared_to(barrel.pos) < hit_range ** 2:
                    lives -= 1
                    player.reset()
                    barrels.clear()
                    state = "play" if lives > 0 else "lose"
                    break
                above = 0 < barrel.pos.y - player.pos.y + BARREL_R < 40
                if not player.on_ground and above and abs(barrel.pos.x - player.pos.x) < 12 and not barrel.scored:
                    barrel.scored = True
                    barrel.points = int(100 * (score_multiplier(score) or 1))
                    score += barrel.points
                    on_barrel_jumped(player, barrel)
            barrels[:] = [b for b in barrels if b.pos.y < HEIGHT + 30]
            if player.center().distance_to(pygame.Vector2(PRINCESS_POS)) < 24:
                score += 1000
                state = "win"
        for f in FLOATERS:
            f[2] += dt
        FLOATERS[:] = [f for f in FLOATERS if f[2] < FLOAT_LIFE]
        draw_scene(screen, font, player, barrels, score, lives, state)
        pygame.display.flip()
    pygame.quit()


if __name__ == "__main__":
    main()
```